Ginko: A Local-First Canonical Memory and Coordination Layer for AI Agents

Separating Conclusions, Evidence, and Agent Interaction State

Rafael Salema Marques
github.com/salemarsm/ginko

White Paper, Version 1.0
October 2026

Abstract

Modern AI agents require persistent memory that survives across sessions, projects, model providers, and increasingly across multiple cooperating agents. Current solutions conflate three distinct concerns—raw conversation history, retrieved document fragments, and durable canonical knowledge—into a single retrieval surface, typically a vector database populated by language-model extractions. This conflation has measurable costs: opaque provenance, weak auditability, weak portability across LLMs, and structural dependence on the same model that produced the memories in the first place. This paper presents *Ginko*, the product distribution of the *ginko* project: a local-first canonical memory layer for AI agents, named after *Ginkgo biloba*, the medicinal plant popularly associated with memory and cognition. Ginko separates events, evidence, canonical conclusions, and agent interaction state into distinct primitives backed by SQLite as the single source of truth. It exposes these primitives through the Model Context Protocol (MCP), HTTP, and CLI tools, and treats embeddings as replaceable indexes rather than primary storage. We describe the data model, retrieval pipeline, supersession lifecycle, audit trail, governance layer (topic-key-driven auto-supersession, confidence-gated approval queues, sensitive-data detection), agent-session continuity, MCP integration, an evidence-aware RAG bridge for document ingestion, a hybrid retrieval ranker, and a persistent inter-agent coordination layer. We then analyze the system against representative alternatives (Mem0, Letta, Zep, Cognee, Engram) and discuss limitations and the remaining roadmap.

Keywords: agent memory, local-first software, multi-agent coordination, retrieval, BM25, SQLite, FTS5, Model Context Protocol, RAG, auditability, supersession.

1. Introduction

Large Language Models (LLMs) are stateless by design. Each call to a model is independent: the model has no intrinsic memory of prior interactions, prior decisions, or prior corrections. When LLMs are embedded into agents—software systems that issue multi-turn requests, manipulate

external state, and persist across sessions—this statelessness becomes a structural problem. Agents must reconstruct context on every turn, either by replaying conversation history, retrieving from external stores, or relying on the user to repeat themselves.

The dominant industry response has been to introduce a memory subsystem alongside the agent: a service that captures conversational fragments, embeds them, stores them in a vector database, and retrieves them by semantic similarity at query time. Variations of this pattern underlie commercial offerings such as Mem0, Zep, Cognee, and the proprietary memory features of consumer assistants. These systems are pragmatic and often effective, but they share a set of architectural assumptions that, in our view, conflate fundamentally different kinds of information.

This paper argues that durable agent memory should be designed around a different primitive: the *canonical memory record*. A canonical memory is a structured, auditable conclusion—not a fragment of dialogue, not a chunk of a document, not a vector. Vectors, document chunks, and raw conversational events have their place; they are evidence, not conclusion. Conflating them collapses provenance, complicates audit, and binds the memory layer to the LLM that produced it.

We present *Ginko*, the user-facing distribution of the *ginko* project, as a working implementation of this thesis. The project name *ginko* describes the technical artifact: an LLM-agnostic memory substrate. The product name *Ginko* is intentionally shorter and more memorable, inspired by *Ginkgo biloba*, the medicinal plant and supplement popularly associated with memory support. The spelling *Ginko* is used as the distribution name, while acknowledging the botanical spelling *Ginkgo*. The system is local-first, single-binary (Go), backed by SQLite with FTS5 full-text search, and integrates with agents through the Model Context Protocol. It is open source, with deliberate scope: a stable canonical core, transparent retrieval, governance, hybrid ranking, RAG bridging, and cooperative multi-agent workflows.

1.1 Contributions

- A canonical memory data model that separates events, evidence, conclusions, and projection layers, with explicit provenance, confidence, scope, and supersession lifecycle.
- A retrieval pipeline that uses BM25 over an FTS5 index for candidate generation, re-ranks candidates with a hybrid `final_score` (lexical, confidence, provenance, recency, and scope components), applies subject-scope fallbacks for conversational queries that miss textual matches, enforces confidence thresholds, and emits auditable retrieval events.
- A feedback loop based on context identifiers and structured feedback events, designed to close the loop between retrieval decisions and downstream usefulness without coupling to any specific tokenizer or evaluation protocol.
- A native Model Context Protocol server that exposes memory operations as transparent agent tools (`memory_context`, `memory_suggest`, `memory_remember`, `memory_search`, `memory_get`, `memory_timeline`, session tools, and coordination-signal tools), with docu-

mented integrations for [CC], Codex-style CLIs, and OpenClaw.

- A product distribution, *Ginko*, that packages the memory system for coding agents and emphasizes one shared local database across heterogeneous LLM clients.
- A persistent inter-agent coordination layer: durable notices, leases, handoffs, blockers, and conflict-review signals shared by agents without turning canonical memory into a high-frequency message queue.
- An open-source reference implementation in Go, with no CGO dependencies, distributable as a single binary across Linux, macOS, and Windows.

2. Background and Motivation

2.1 The conflation problem

Most production agent memory systems treat the following as substitutable forms of "memory":

- Raw conversational history—the literal sequence of user and assistant turns.
- Retrieved document chunks—segments of source documents surfaced via semantic search.
- Canonical durable knowledge—stable preferences, decisions, facts that should govern future agent behavior.

These categories have different lifecycles, different provenance models, and different appropriate retrieval strategies. A user preference ("prefers concise technical answers") should outlive any specific conversation in which it was expressed. A document chunk is meaningful only as evidence for a claim, with the document itself as primary source. A conversational turn is mostly transient—useful for short-term coherence, rarely useful as durable knowledge.

Conflating them has measurable consequences. When all three are stored as embeddings in the same vector index, the agent cannot distinguish "I should follow this rule" from "this is something I once said" from "this is a citation I might quote." When all three are extracted via the same LLM pass at write time, provenance collapses to "the model thought this was important." When the user wishes to inspect, correct, or audit the memory, they encounter an opaque vector store rather than a structured record.

2.2 Three illustrative limitations of vector-first memory

We identify three concrete limitations that motivated the design of *ginko*.

Audit opacity. When a vector-first memory system "remembers" an incorrect or stale fact, the user cannot easily answer questions such as: which conversation produced this memory? when was it created? was it superseded? what was its confidence? Vector stores typically expose embeddings and payloads, but the curation logic that decided to write the memory in the first place is not preserved as a first-class object.

Operational weight. Self-hosting common alternatives requires PostgreSQL with pgvector, a separate vector database (Qdrant, Weaviate, Milvus), or a managed service. For developers who want to run a memory layer alongside agents on a workstation, a Raspberry Pi, or a containerized environment, this footprint is disproportionate to the value delivered.

Vendor coupling. Memory features built into specific assistants (proprietary memory in commercial chat products) are not portable. A user who switches between Claude Code, Codex-like CLIs, and locally hosted models cannot carry their durable preferences and project decisions across these tools.

2.3 The local-first principle

Local-first software, as articulated by Kleppmann *et al.* [2], emphasizes user agency, longevity of data, and resilience to network or vendor failure. Memory for AI agents is a natural fit for this principle: the data being stored represents the user's preferences, their projects' decisions, and their working knowledge. It is precisely the class of data that should not be hostage to a third-party service. *ginko* is designed so that the canonical store—a single SQLite file—is fully owned, inspectable, and portable by the user.

3. Design Principles

The system is governed by five principles, in priority order.

3.1 Memory is conclusion; RAG is evidence

A canonical memory is a curated assertion: "the user prefers Go code examples"; "this project uses SQLite as the local store." A document chunk, by contrast, answers a different question: "where did this claim come from?" The two should not share a representation. *ginko* stores canonical memories as structured records and stores supporting evidence as separate document and chunk tables, with optional links between them.

3.2 SQLite is the source of truth

The single SQLite database file is authoritative. All other representations—FTS5 indexes, embedding references, projected context strings—are derived. This invariant has practical consequences: the system can be inspected with standard `sqlite3` tooling, backed up by file copy, version-controlled if the user wishes, and migrated by SQL.

3.3 Embeddings are optional indexes

Vector embeddings are useful as one signal in retrieval, but they are not canonical memory. The schema reflects this by storing embeddings as references (`embedding_refs`) rather than as columns of vector type. A future change of embedding provider or model should not invalidate the memory store.

3.4 The LLM is a client, not a database

Many systems delegate memory curation to the LLM itself: the model summarizes conversations, extracts facts, and writes them back to memory. In *ginko*, the LLM may suggest candidates—via the `memory_suggest` tool—but the durable write is a deliberate operation, not an automatic side effect of inference. This decouples the memory layer from the model's reliability and biases.

3.5 Auditability is a feature, not an afterthought

Every meaningful operation—memory creation, supersession, deletion, context build, context feedback—is recorded as an append-only event. The schema enforces non-destructive supersession (old memories carry a `superseded_by` reference rather than being overwritten). This design makes the system suitable for environments where forensic-grade traceability is required, while remaining lightweight enough for casual personal use.

4. System Architecture

4.1 Component overview

The reference implementation is composed of a product CLI, lower-level binaries, and a shared core library:

- **ginko**—the user-facing distribution CLI. It starts MCP and HTTP services, saves/searches/contextualizes memories, runs diagnostics, and configures agent integrations.
- **memserver**—HTTP server with OpenAPI-documented endpoints and an embedded local web GUI.
- **memmcp**—Model Context Protocol server speaking JSON-RPC over stdio.
- **memctl**—Command-line client for direct interaction with the HTTP API.
- **ginko-admin**—administrative utility for project initialization, diagnostics, and configuration generation.

The distribution binaries link the same `memory` package, which contains the data model, store operations, retrieval, and context projection. The system has no required external dependencies beyond Go 1.22 and uses `modernc.org/sqlite` to avoid CGO.

4.2 Data model

The canonical memory record is the central entity:

```
type Memory struct {
 ID string
 Type MemoryType // preference, fact, decision, task, note, relationship
 Subject string // who or what the memory is about
 Content string
```

```
 Source Source // provenance: kind + ref
 Scope Scope // global, project, session, private
 Confidence float64 // [0, 1], CHECK-enforced
 CreatedAt time.Time
 UpdatedAt time.Time
 ValidFrom *time.Time
 ValidUntil *time.Time
 SupersedesID *string
 SupersededBy *string
 TopicKey string // auto-supersession key within subject+scope
 Status MemoryStatus // active, pending, deleted
 Tags []string
 EmbeddingRefs map[string]string
}
```

Each field encodes a design decision. `Type` and `Scope` make the memory's role and visibility explicit. `Source` preserves provenance: a memory without provenance is, in this model, weak memory. `Confidence` is bounded and validated at the schema level. `SupersedesID` and `SupersededBy` implement non-destructive replacement. `EmbeddingRefs` is a string map rather than a vector column, enforcing the "embeddings are external indexes" principle.

Two auxiliary tables—`Documents` and `Chunks`—provide the evidence layer for RAG-style use cases, each with FTS5 indexes for full-text search. A `ingestion_runs` table tracks every ingest operation with SHA-256 deduplication, enabling re-ingest detection and citation provenance. A `context_feedback` table stores feedback on retrieval decisions. A `sessions` table records project-scoped work sessions, `agent_signals` holds inter-agent coordination state, and `retrieval_eval_runs` together with `retrieval_eval_items` persist retrieval evaluations for historical comparison. A `schema_migrations` table records applied migrations. All timestamps are stored as RFC3339Nano strings for portability.

4.3 Storage layer

The store opens SQLite with WAL journaling enabled and a single open connection (`SetMaxOpenConns(1)`) to avoid `SQLITE_BUSY` errors under concurrent writes from a single binary. Migrations are idempotent and version-tagged. IDs are UUIDs prefixed by entity type (`mem_`, `evt_`, `doc_`, `chk_`, `ctx_`, `cfb_`), avoiding the time-based collisions of an earlier design and providing predictable inspection.

Tags are stored both as a JSON column on the memory record (for round-trip API fidelity) and in a normalized `memory_tags` table (for performant filtering). This duplication is intentional: the JSON column preserves the user's view of the record, while the normalized table supports indexed queries such as "all memories tagged X within scope Y."

4.4 Retrieval pipeline

Retrieval is the operation by which a prompt or query is converted into a compact, prompt-ready set of canonical memories. The pipeline has six logical stages:

1. **Query normalization.** Conversational queries are tokenized using a Unicode-aware regular expression (`\pL\pN_`), lowercased, filtered against a small Portuguese/English stopword list, and recombined as an FTS5 OR-of-prefixes expression. Short tokens are retained when they appear to be technical (containing digits or uppercase letters), preserving terms such as "AI", "Go", "v8", "k8s".

2. **Candidate generation.** The normalized query is matched against the `memories_fts` virtual table. Results are ordered by BM25 score (ascending, since SQLite's BM25 implementation returns lower-is-better), with ties broken by confidence and recency.

3. **Subject fallback.** If FTS yields no candidates and a subject is specified, the system falls back to retrieving recent high-confidence memories for that subject. This handles the conversational case where the user prompt shares no surface tokens with the durable memories that should govern the response.

4. **Hybrid ranking.** Candidates are re-sorted by a `final_score` that combines several signals rather than trusting raw BM25 order. When a lexical match is present, `final_score` is $0.40 \cdot \text{lexical} + 0.28 \cdot \text{confidence} + 0.13 \cdot \text{provenance} + 0.09 \cdot \text{recency} + 0.10 \cdot \text{scope}$. On the subject-fallback path, where no lexical score exists, the lexical term is dropped and the remaining weights are $0.40 \cdot \text{confidence} + 0.30 \cdot \text{provenance} + 0.20 \cdot \text{recency} + 0.10 \cdot \text{scope}$. The scope component rewards specificity: project = 1.0, session = 0.8, global = 0.6, otherwise 0.4. Each selected memory carries a `RankingMetadata` record with the individual component scores and a human-readable `rank_reason`.

5. **Filtering.** Superseded memories are excluded by SQL. Memories with confidence below 0.5 are excluded from automatic context injection, regardless of textual relevance. Scope and type filters are applied as supplied.

6. **Budget packing.** A greedy fill respects a token budget (default 1,200, capped at 8,000), estimated using a deterministic rune-based approximation. Memories that exceed the remaining budget cause the response to be marked truncated.

The result is wrapped in a `ContextResponse` that includes a `context_id`, the projected text, the selected memory records, per-memory ranking metadata (keyed by memory ID), budget metadata, and—for RAG-sourced memories—chunk citations. The response also carries a `conflicts` field: when two or more active memories in the same result share a subject and type, the pipeline reports them as a structural conflict group rather than silently choosing a winner. This is a cheap invariant check, not semantic contradiction detection; `topic_key` supersession should normally prevent such duplicates, but when it does not, the caller is told. A `context.built` event is appended to the audit log when at least one memory is selected.

4.5 Agent sessions and multi-agent portability

Ginko treats LLMs and agent runtimes as clients of the same canonical database. Claude Code, OpenClaw, Codex-style CLIs, and local model runners can therefore share durable preferences, project decisions, and session summaries without depending on a specific vendor's memory feature. Session primitives record project-scoped starts, endings, and summaries so that a future session can recover compact continuity without replaying the full chat transcript.

This portability is more than convenience. It is a governance boundary: no single model owns the user's memory. Different agents may contribute records, but every durable record keeps provenance, scope, confidence, timestamps, and lifecycle state. The database remains inspectable with ordinary SQLite tools, and the MCP layer remains an integration protocol rather than a hidden store.

4.6 Feedback loop

The system instruments retrieval to permit closed-loop evaluation without committing to any single ranking algorithm. Each `ContextResponse` carries a `context_id`. Clients (or the user, via the GUI) may submit feedback through `POST /api/feedback` referencing this id, indicating whether the retrieved set was useful and which specific memories were ultimately incorporated. Feedback is stored in the `context_feedback` table and emitted as a `context.feedback` event.

The feedback signal is not currently used to alter ranking. It is collected so that future versions can perform retrospective evaluation, A/B testing of ranking strategies, and supervised tuning of confidence thresholds and decay parameters.

A retrieval evaluation harness complements this loop. `RunEval` executes annotated fixtures against the hybrid ranker and computes precision@5 and nDCG@10 per fixture, persisting each run and its per-item rankings in the `retrieval_eval_runs` and `retrieval_eval_items` tables for historical comparison. The fixtures cover lexical precision, scope priority, supersession exclusion, conflict detection, and token-budget enforcement. Like the feedback stream, evaluation results are recorded for comparison and do not yet feed back into ranking automatically.

4.7 Integration surfaces

Three integration surfaces share the same canonical model:

- **HTTP API.** JSON over HTTP with OpenAPI documentation. Endpoints exist under both `/api/*` (legacy) and `/api/v1/*` (versioned), enabling forward compatibility. A `/healthz` endpoint supports orchestration. The HTTP server enforces `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, and `IdleTimeout` to resist slow-loris-style abuse even on loopback.
- **Model Context Protocol.** A JSON-RPC server over stdio, conformant to MCP protocol version 2024-11-05 [1]. The MCP surface exposes token-budgeted context retrieval (`memory_context`), candidate generation (`memory_suggest`), durable writes (`memory_remember`), raw search (`memory_search`), full-record retrieval (`memory_get`), lifecycle audit (`memory_timeline`), session continuity tools (`memory_session_start`, `memory_session_end`, `memory_session_summary`),

and coordination-signal tools (`signal_create`, `signal_list`, `signal_update`)—twelve tools in total. The buffer is configured for payloads up to 10 MiB.

- **CLI.** `ginko` is the recommended end-user CLI. `memctl` remains available for scriptable lower-level access to primary operations and non-MCP automation. `ginko-admin` provides administrative utilities for initialization, diagnostics, and configuration.

4.8 Persistent inter-agent coordination

As agent workflows become multi-actor, memory alone is not enough. Two agents may need to know that one is editing a file, that a task is blocked, that a review is requested, or that a previous agent is handing off unfinished work. These are not canonical facts about the world; they are interaction state. Ginko therefore treats persistent inter-agent coordination as a related but separate layer.

The coordination primitive is an `agent_signals` table backed by the same local SQLite and surfaced through MCP. A signal has a project, topic key, kind, lifecycle status, owner agent, optional target agent, payload, creation time, and optional expiry. Signal kinds include:

- **notice**—a durable announcement relevant to other agents.
- **lease**—a soft lock over a file, topic, task, or decision, always with `expires_at`; leases must never be infinite.
- **handoff**—a structured "agent A stopped here; agent B should continue there" record.
- **conflict**—a reviewable contradiction or competing claim surfaced by memory governance.
- **review_request**—a request for another agent or human to inspect a change or memory candidate.
- **blocker**—a durable explanation of why progress is paused.

This design deliberately does not turn Ginko into a message bus. High-frequency chat, real-time pub/sub, and strong distributed locking remain non-goals. The coordination layer is for durable, low-frequency workflow state that should survive process restarts and be visible to heterogeneous agents sharing the same local memory substrate.

The layer is implemented. Signals move through an explicit lifecycle (`active`, `acknowledged`, `expired`, `resolved`, `cancelled`); leases carry a mandatory `expires_at`, and a bulk `ExpireStaleSignals` operation sweeps active signals whose expiry has passed. Signals are created, listed, and transitioned through the MCP tools `signal_create`, `signal_list`, and `signal_update`, through REST endpoints for creation, listing, status transitions, and expiry sweeps, and through a dedicated Signals view in the embedded GUI. The coordination layer is also wired into memory governance: when `memory_remember` finds contradicting active memories, it records a `conflict` signal automatically, so a detected contradiction becomes visible coordination state rather than a silent overwrite.

5. Implementation Notes

5.1 Scale and shape of the codebase

The implementation is deliberately small relative to the problem it addresses. The core remains a Go codebase organized around a memory package, HTTP server, MCP server, and CLI distribution. A reader can audit the essential system behavior in a single sitting. This is itself a feature: the smaller the surface, the easier it is to reason about correctness, performance, and security.

5.2 Schema migrations

Migrations are stored in code as a list of idempotent SQL statements applied at store open. A `schema_migrations` table records the applied version. `ALTER TABLE` statements that have already been applied are tolerated via duplicate-column detection. This pattern is sufficient for the current scope and avoids the operational burden of a migration framework. Migrations are forward-only: the documented upgrade path is to stop the server, back up the database, install the new binary, and let migrations run on startup. From v1.0, schema changes are additive—new tables and new nullable columns—while breaking changes require a new major version. The schema version is reported by `ginko doctor` and the `/api/config` endpoint.

5.3 Concurrency and durability

WAL journaling is enabled to permit concurrent reads alongside writes. The single open connection serializes writes within a process, which is appropriate for the single-user local-first deployment model. Multi-writer scenarios are explicitly out of scope for the current release line.

5.4 Packaging and distribution

The project ships as pre-built archives for Linux, macOS, and Windows (amd64 and arm64), plus a container image. GoReleaser builds each binary with `CGO_ENABLED=0` and stamps version, commit, and build date via linker flags. A multi-stage `Dockerfile` produces a distroless/static image that exposes port 8787, mounts `/data` as a volume, and reads the database path from `GINKO_DB`; GoReleaser publishes `linux/amd64` and `linux/arm64` images to `ghcr.io/salemarsm/ginko` and unifies them under a multi-arch manifest. Installation documentation covers each operating system, the container path, and building from source. A Homebrew formula is not yet provided; it remains on the backlog.

5.5 Testing posture

Tests focus on invariants rather than implementation details. Representative cases include: that BM25-ranked preferences win over recent weakly-related tasks; that subject-scope fallback recovers durable memories when FTS misses; that low-confidence memories never enter automatic context; that empty contexts do not pollute the audit log; and that technical short tokens (e.g., “AI”, “k8s”) survive query normalization. Retrieval evaluation fixtures add coverage for supersession filtering, scope priority, and token-budget enforcement, and signal tests exercise

the coordination lifecycle. These tests are intended to remain valid as the underlying ranking strategy evolves.

6. Comparison with Related Systems

A useful frame for situating *ginko* is a four-quadrant matrix along two axes: storage canonicality (vector-first vs structured-first) and deployment posture (hosted vs local-first). Most active systems occupy three of the four quadrants. The local-first, structured-first quadrant is comparatively underpopulated, and that is where *ginko* situates itself.

System	Primary store	Local-first	Audit model
Mem0 [7]	Vector DB + LLM extractions	Hosted-first	Inferred (LLM extracted)
Letta (MemGPT) [6]	Agent-internal memory tiers	Hosted or self-hosted	Coupled to agent runtime
Zep [8]	PostgreSQL + pgvector + graph	Self-hostable, heavyweight	Conversational fact extraction
Cognee [9]	Knowledge graph + vector	Multi-backend	LLM-driven graph construction
Engram	SQLite + FTS5 + MCP	Local-first	Session/observation oriented
ginko [10]	SQLite + FTS5 + events	Local-first	Canonical, append-only events, supersession

Table 1: Comparison of agent memory systems along storage and audit dimensions.

6.1 Engram

Engram is the closest known peer in technical posture: Go, SQLite, FTS5, MCP. The projects differ primarily in lens: Engram emphasizes coding-agent observations and session memory; *ginko* emphasizes a canonical lifecycle (events, evidence, conclusions, supersession, projection) intended to support coding agents but to generalize beyond them. The systems can be complementary, with Engram serving as a coding-agent surface and *ginko* serving as a longer-term canonical store.

6.2 Mem0, Zep, Cognee

These systems are mature and offer features (managed hosting, conversational extraction pipelines, multi-tenant isolation) that *ginko* does not. They are appropriate choices when those features are needed. They do not, however, prioritize local-first deployment, single-binary distribution, or canonical separation of evidence from conclusion in the way *ginko* does. For users for whom auditability and ownership outweigh convenience, the tradeoff favors a structured-first, local-first design.

6.3 Vendor-coupled assistant memory

Memory features built into commercial chat assistants are convenient but not portable across tools, models, or vendors. A user who works with multiple agents (a coding CLI, a generic chat assistant, a local model) will find that none of these built-in memories share state. A neutral memory layer—accessed via MCP—makes durable preferences and decisions portable across this fragmented landscape.

7. Limitations and Threats to Validity

We summarize the principal limitations of the current implementation honestly. These are reflected in the public roadmap and backlog.

- **Heuristic suggestion engine.** `memory_suggest` currently uses a keyword-based cascade with hardcoded Portuguese and English triggers. This is sufficient for proof of concept and for a "human in the loop" workflow where every candidate requires confirmation, but it is not the intended long-term architecture. The roadmap calls for a `Suggester` interface with pluggable implementations, including LLM-backed suggesters.

- **Authentication remains local-first and deployment-sensitive.** The system is designed primarily for trusted local use. Bearer-token support and loopback-oriented defaults reduce accidental exposure, but production multi-user isolation remains out of scope. The intended protection model is a single-user local, VPN, or container deployment.

- **Token estimation.** `EstimateTokens` uses a deterministic rune-count heuristic ($\lfloor n/4 \rfloor + 1$). This is acceptable for budgeting but diverges from real BPE tokenizers, particularly for non-Latin scripts. A pluggable tokenizer interface is on the roadmap.

- **Single-tenant assumption.** The store assumes a single trusted user. There is no row-level security, no per-user encryption, and no scoped credential model. Multi-tenant deployments are not supported and not currently a goal.

- **Embeddings are an extension point, not a bundled capability.** Hybrid ranking is implemented across lexical, confidence, provenance, recency, and scope signals, but the semantic component stays optional: the store exposes an `EmbeddingAdapter` interface that a host may wire in, and no embedding provider ships with the distribution. Retrieval therefore runs on lexical signals by default and cannot resolve purely paraphrastic matches until an adapter is configured.

- **Contradiction resolution is partial.** Topic-key-driven auto-supersession, pending approval queues, sensitive-data detection, and structural conflict detection are implemented: the context response reports memory groups that share a subject and type, and `memory_remember` opens a `conflict` signal when it finds contradicting active memories. What is not implemented is automatic resolution—confidence reinforcement, merging, or model-assisted adjudication of competing claims. Detection surfaces the ambiguity; a

human or an agent still has to decide.

- **Inter-agent coordination is durable state, not a message bus.** The signal layer supports notices, leases, handoffs, blockers, and review requests, but it is not intended for high-frequency chat or strong distributed locking.
- **Soft-delete semantics are implemented but lifecycle-aware retrieval is still maturing.** The `deleted` status and tombstone behavior are available. Lifecycle-aware retrieval policies (e.g., auto-expiry, tombstone propagation) remain on the roadmap. Backup of the SQLite file remains the recommended safety practice.

8. Roadmap

The project follows a versioned roadmap with explicit exit criteria for each release.

- **v0.1 — Local core.** *(Implemented.)* Canonical store, FTS5 retrieval, MCP server, GUI, and the first working local memory model.
- **v0.2 — Local safety and installability.** *(Implemented.)* API tokens, auth-aware MCP propagation, install scripts, structured logging, smoke tests, and safer local setup.
- **v0.3 — Agent integration quality.** *(Implemented.)* Tested integrations for Claude Code, Codex-style CLIs, OpenClaw, and generic MCP clients. Progressive-disclosure retrieval (`memory_get`, `memory_timeline`, and session tools). One-command setup with dry-run.
- **v0.4 — Governance and memory quality.** *(Implemented.)* Topic-key-driven auto-supersession, status/lifecycle fields (`active`/`pending`/`deleted`), soft delete, sensitive-data detection, approval queues, audit GUI, and conflict detection primitives.
- **v0.5 — RAG bridge.** *(Implemented.)* Document ingestion with SHA-256 deduplication, chunk evidence, traceable ingestion runs with re-ingest status, candidate extraction from evidence, and citation-aware context.
- **v0.6 — Retrieval quality.** *(Implemented.)* Hybrid ranking by `final_score` (lexical, confidence, provenance, recency, and scope components), per-memory ranking metadata, structural conflict detection in context responses, an optional `EmbeddingAdapter` extension point, and a retrieval evaluation harness computing precision@5 and nDCG@10 over annotated fixtures. Two originally scoped items remain open: MMR-style diversification and subject-hierarchy aware retrieval.
- **v0.6 (cross-cutting) — Persistent inter-agent coordination.** *(Implemented.)* The `agent_signals` table and its status lifecycle, the `signal_create`, `signal_list`, and `signal_update` MCP tools, REST endpoints for creation, status transitions, and stale-lease expiry, a dedicated GUI view, and automatic `conflict` signals raised by `memory_remember`.

- **v0.7 — Packaging and distribution.** *(Implemented.)* GitHub release artifacts with checksums, a multi-stage `Dockerfile` and a multi-arch image on `ghcr.io`, per-operating-system installation documentation, a contribution guide, issue templates, and a changelog. A Homebrew formula remains open.

- **v1.0 — Stable local-first memory server.** *(Implemented.)* OpenAPI contract advanced to 1.0.0 with the complete endpoint and schema set, a documented schema-migration strategy, documented backup/restore and a written security model, and a stable MCP tool contract. Multi-user production isolation remains out of scope.

9. Conclusion

AI agents need memory, but not all memory is alike. Conflating raw conversation, retrieved evidence, durable canonical knowledge, and workflow coordination into a single vector index simplifies implementation at the cost of audit, ownership, and clarity. *Ginko* is an argument, in code, that a different design is feasible: a small, local-first, structured-first system in which canonical memories are first-class, evidence is preserved separately, coordination state is explicit, embeddings are replaceable, and the LLM is a client rather than a database.

The implementation demonstrates that this design is practical: a Go codebase, a single SQLite source of truth, an MCP-native integration story, and a retrieval pipeline whose behavior is testable, auditable, and inspectable. The Ginko distribution makes that substrate approachable for real agent workflows, and the governance, hybrid retrieval, evidence-aware RAG, and persistent inter-agent coordination layers described above are implemented rather than aspirational. The roadmap is now honest about a smaller set of open items: automatic contradiction resolution, production multi-user isolation, a bundled embedding provider, MMR-style diversification, and a Homebrew formula. The project is open source under the MIT license at github.com/salemarsm/ginko and welcomes contributions, scrutiny, and disagreement.

References

[1] Anthropic. Model Context Protocol Specification, 2024-11-05 revision. `https://modelcontextprotocol.io`.

[2] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proc. Onward!*, 2019.

[3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[4] D. R. Hipp. SQLite FTS5 extension documentation. `https://www.sqlite.org/fts5.html`.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for

knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[7] Mem0 documentation. https://docs.mem0.ai.

[8] Zep documentation. https://www.getzep.com/docs.

[9] Cognee documentation. https://docs.cognee.ai.

[10] *ginko* source repository. https://github.com/salemarsm/ginko.

A. Project Status Snapshot (October 2026)

- Distribution name: Ginko, inspired by *Ginkgo biloba*, the medicinal plant popularly associated with memory.
- Technical project/repository name: `ginko`
- Latest released milestone: v1.0 (stable local-first memory server), following v0.6 (retrieval quality and agent coordination) and v0.7 (packaging and distribution).
- License: MIT
- Language: Go 1.22+
- Dependencies: `modernc.org/sqlite`, `google/uuid` (no CGO)
- Implemented features: canonical memory store, FTS5 retrieval with a hybrid `final_score` ranker and per-memory ranking metadata, structural conflict detection, a retrieval evaluation harness, an MCP server (12 tools) including coordination-signal tools, HTTP API with a 1.0.0 OpenAPI contract, embedded GUI, session continuity, governance layer (topic-key supersession, approval queues, sensitive-data detection), persistent inter-agent coordination signals, and RAG document ingestion with SHA-256 deduplication and citation context.
- Test coverage: invariant-focused unit, integration, MCP contract, retrieval-evaluation, and smoke tests
- Distribution: `ginko` CLI plus `memctl`, `memmcp`, `memserver`, and `ginko-admin` binaries; pre-built archives for Linux, macOS, and Windows (amd64/arm64) and a multi-arch container image at `ghcr.io/salemarsm/ginko`
- Not yet implemented: automatic contradiction resolution, production multi-user isolation, a bundled embedding provider, and a Homebrew formula.